University of Engineering & Management, Kolkata
Institute of Engineering and Management, Newtown Campus
Department of Computer Applications

ASSIGNMENT SUBMISSION — TOP SHEET

Technical Class Assessment — April 2026
Database Management Systems & Java with JDBC | MCA Programme

STUDENT DETAILS

Full Name	Arpit Awasthi		
Roll Number	18	Registration No.	304202500600018
Section	A	Semester	2
Programme	MCA	Academic Year	2025 - 2026
Email ID	arpit.awasthi2025@iem.edu.in		

ASSIGNMENT DETAILS

Subject	Java – DBMS Assignment	Set Number	4
Problem Title	The Hostel Register		
Language Used	DBMS & Core Java	Database Used	MySQL 8.0.44
Submission Date	10th April 2026	Total Pages	11

SUBMISSION CHECKLIST

#	Item	Included (✓ / ✗)	Pages / Remarks
1	Printed Java Source Code (DBConnection.java)		
2	Printed Java Source Code (Main.java)		
3	Printed SQL Schema File (schema.sql)		
4	Screenshot(s) of Program Execution (printed)		
5	Problem Statement (this top sheet attached)		

DECLARATION

I hereby declare that this assignment is my own original work. I have not copied from any other student or used any unauthorised sources. I understand that plagiarism will result in zero marks and disciplinary action.

Student Signature & Date

Faculty Signature & Date

TECHNICAL CLASS ASSESSMENT — April – 2026

Degree: MCA	Stream: MCA
Due Date: _____	**Set No: 4**

🏠 The Hostel Register

[Domain: Campus Administration]

Language	Core Java (Console Application)
Database	MySQL
Connectivity	JDBC (Java Database Connectivity)

The Story

> *"Three students claim Room 204. I have no idea who I gave it to last month."*

The warden of a college hostel in Kolkata tracks room allotments in a worn notebook. Students come and go, rooms are double-booked, and the warden never knows how many beds are vacant at any given time. During the admission rush, the situation becomes chaotic.

Your task is to build a Hostel Room Allotment System using Java and MySQL.

Your Task

Build a Hostel Room Allotment System as a Java console application using JDBC and MySQL.

Your system must allow:

- Add a room with room number, capacity, and floor
- Allot a room to a student (check if capacity allows it)
- Vacate a student from a room
- Display all rooms with their current occupancy count and available seats

Database Schema

Create the following table(s) in MySQL:

```
CREATE TABLE rooms (
    room_id   INT PRIMARY KEY AUTO_INCREMENT,
    room_no   VARCHAR(10) UNIQUE,
    floor     INT,
    capacity  INT
);

CREATE TABLE allotments (
```

```
    allot_id   INT PRIMARY KEY AUTO_INCREMENT,
    room_id    INT,
    student    VARCHAR(100),
    allot_date DATE DEFAULT (CURRENT_DATE),
    vacated    BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

Required Features

Feature	Implementation Hint
Add Room	INSERT into rooms with room_no, floor, capacity
Allot Room	Check COUNT of active allotments < capacity, then INSERT
Vacate Student	UPDATE allotments SET vacated=TRUE WHERE student=? AND room_id=?
Room Status	SELECT room_no, capacity, COUNT active allotments, available seats

Constraints

- Use Core Java only — no Spring, Hibernate, or web frameworks
- Connect to MySQL using JDBC (no JPA / ORM)
- All operations must be menu-driven via console
- No GUI required — Scanner-based input is sufficient
- Handle SQL exceptions with try-catch blocks
- Use PreparedStatement for all SQL queries

Sample Interaction

```
----- Hostel Management -----
1. Add Room
2. Allot Room to Student
3. Vacate Student
4. View Room Status
5. Exit
Enter choice: 4
Room 101 | Floor 1 | Capacity 3 | Occupied 2 | Available 1
Room 204 | Floor 2 | Capacity 2 | Occupied 2 | Available 0 [FULL]
```

Submission Checklist

- DBConnection.java (JDBC connection class)
- Main.java (menu-driven main program)
- SQL file: schema.sql (CREATE TABLE statements)
- screenshot.png (terminal output showing at least 2 features working)

> *"A database is not just data. It is the memory of your application. Make it reliable."*

Main.Java

```
import java.sql.*;
import java.util.Scanner;

public class Main {

    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {

        while (true) {

            System.out.println("\n===== Hostel Management =====");
            System.out.println("1. Add Room");
            System.out.println("2. Allot Room to Student");
            System.out.println("3. Vacate Student");
            System.out.println("4. View Room Status");
            System.out.println("5. Exit");

            System.out.print("Enter choice: ");
            int choice = sc.nextInt();

            switch (choice) {

                case 1:
                    addRoom();
                    break;

                case 2:
                    allotRoom();
                    break;

                case 3:
                    vacateStudent();
                    break;

                case 4:
                    viewRoomStatus();
                    break;

                case 5:
                    System.out.println("Thank you!");
                    System.exit(0);

                default:
                    System.out.println("Invalid  choice!");
            }
        }
    }

    // ✅ Add Room
    static void addRoom() {
        try (Connection con = DBConnection.getConnection()) {

            System.out.print("Enter Room No: ");
            String roomNo = sc.next();

            System.out.print("Enter Floor: ");
            int floor = sc.nextInt();

            System.out.print("Enter Capacity:  ");
```

```
        int capacity = sc.nextInt();

        String query = "INSERT INTO rooms(room_no, floor, capacity) VALUES (?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(query);

        ps.setString(1, roomNo);
        ps.setInt(2, floor);
        ps.setInt(3, capacity);

        ps.executeUpdate();
        System.out.println("Room added successfully!");

    } catch (Exception e) {
        System.out.println("Error adding room!");
        e.printStackTrace();
    }
}

// ✅ Allot Room
static void allotRoom() {
    try (Connection con = DBConnection.getConnection()) {

        System.out.print("Enter Room No: ");
        String roomNo = sc.next();

        System.out.print("Enter Student Name: ");
        sc.nextLine();
        String student = sc.nextLine();

        // Get room details
        String q1 = "SELECT room_id, capacity FROM rooms WHERE room_no=?";
        PreparedStatement ps1 = con.prepareStatement(q1);
        ps1.setString(1, roomNo);
        ResultSet rs = ps1.executeQuery();

        if (!rs.next()) {
            System.out.println("Room not found!");
            return;
        }

        int roomId = rs.getInt("room_id");
        int capacity = rs.getInt("capacity");

        // Count occupants
        String q2 = "SELECT COUNT(*) FROM allotments WHERE room_id=? AND vacated=FALSE";
        PreparedStatement ps2 = con.prepareStatement(q2);
        ps2.setInt(1, roomId);
        ResultSet rs2 = ps2.executeQuery();
        rs2.next();

        int occupied = rs2.getInt(1);

        if (occupied >= capacity) {
            System.out.println("Room is FULL!");
            return;
        }

        // Insert allotment
        String q3 = "INSERT INTO allotments(room_id, student) VALUES (?, ?)";
        PreparedStatement ps3 = con.prepareStatement(q3);
        ps3.setInt(1, roomId);
        ps3.setString(2, student);
```

```
            ps3.executeUpdate();
            System.out.println("Room allotted successfully!");

        } catch (Exception e) {
            System.out.println("Error allotting room!");
            e.printStackTrace();
        }
    }

    // ✅ Vacate Student
    static void vacateStudent() {
        try (Connection con = DBConnection.getConnection()) {

            System.out.print("Enter Room No: ");
            String roomNo = sc.next();

            System.out.print("Enter Student Name: ");
            sc.nextLine();
            String student = sc.nextLine();

            String q1 = "SELECT room_id FROM rooms WHERE room_no=?";
            PreparedStatement ps1 = con.prepareStatement(q1);
            ps1.setString(1, roomNo);
            ResultSet rs = ps1.executeQuery();

            if (!rs.next()) {
                System.out.println("Room not found!");
                return;
            }

            int roomId = rs.getInt("room_id");

            String q2 = "UPDATE allotments SET vacated=TRUE WHERE student=? AND room_id=? AND
vacated=FALSE";
            PreparedStatement ps2 = con.prepareStatement(q2);
            ps2.setString(1, student);
            ps2.setInt(2, roomId);

            int rows = ps2.executeUpdate();

            if (rows > 0) {
                System.out.println("Student vacated successfully!");
            } else {
                System.out.println("No active allotment found!");
            }

        } catch (Exception e) {
            System.out.println("Error vacating student!");
            e.printStackTrace();
        }
    }

    // ✅ View Room Status
    static void viewRoomStatus() {
        try (Connection con = DBConnection.getConnection()) {

            String query = "SELECT r.room_no, r.floor, r.capacity, " +
                    "COUNT(a.allot_id) AS occupied " +
                    "FROM rooms r LEFT JOIN allotments a " +
                    "ON r.room_id = a.room_id AND a.vacated=FALSE " +
                    "GROUP BY r.room_id";
```

```
            PreparedStatement ps = con.prepareStatement(query);
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                String roomNo = rs.getString("room_no");
                int floor = rs.getInt("floor");
                int capacity = rs.getInt("capacity");
                int occupied = rs.getInt("occupied");

                int available = capacity - occupied;

                System.out.print("Room " + roomNo +
                        " | Floor " + floor +
                        " | Capacity " + capacity +
                        " | Occupied " + occupied +
                        " | Available " + available);

                if (available == 0) {
                    System.out.print(" [FULL]");
                }

                System.out.println();
            }

        } catch (Exception e) {
            System.out.println("Error fetching room status!");
            e.printStackTrace();
        }
    }
}
```

src/DBconnection.java

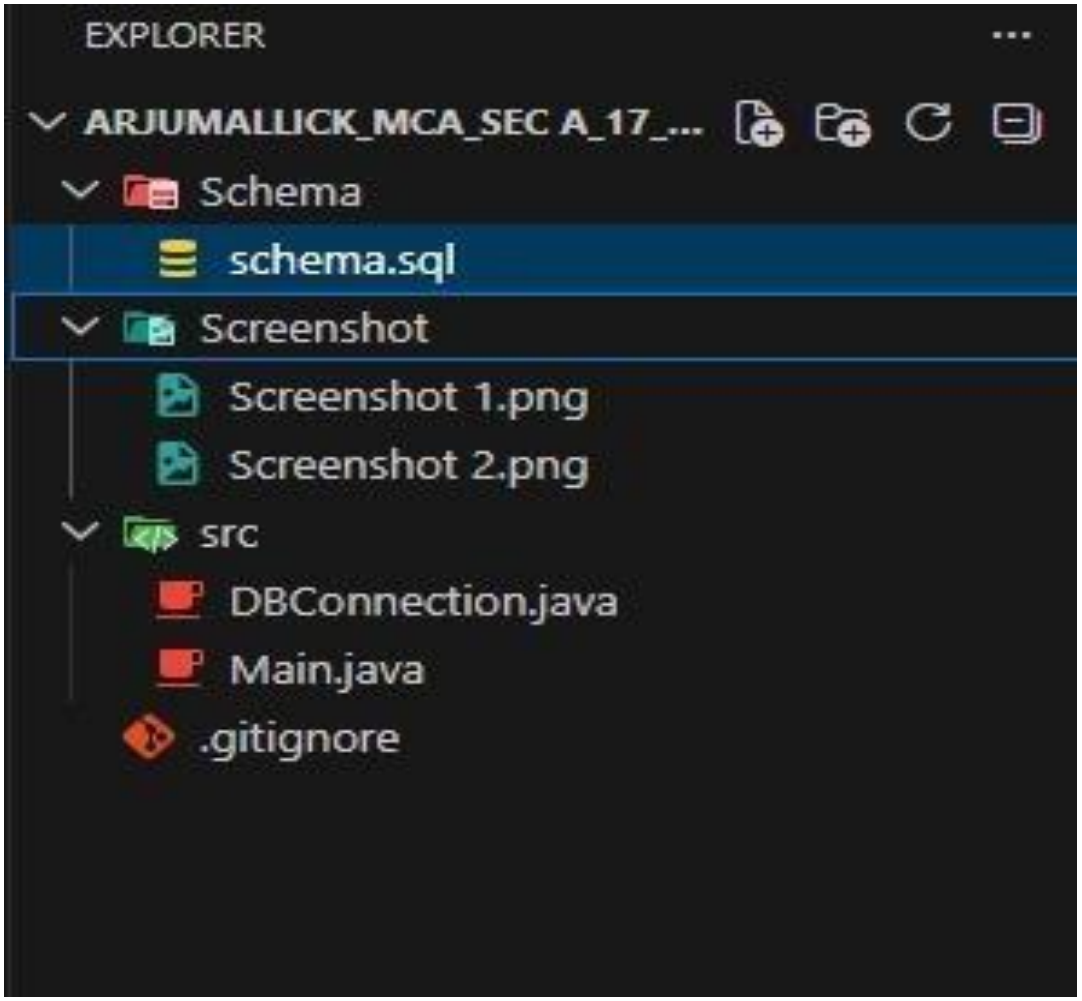
```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/hostel_db";
    private static final String USER = "root";
    private static final String PASSWORD = "Arpit@2005";

    public static Connection getConnection() throws SQLException {
        try {
            //  LOAD DRIVER (FIX)
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL Driver not found!");
            e.printStackTrace();
        }

        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

schema.sql

```sql
CREATE DATABASE hostel_db;
USE hostel_db;

CREATE TABLE rooms (
    room_id   INT PRIMARY KEY AUTO_INCREMENT,
    room_no   VARCHAR(10) UNIQUE,
    floor     INT,
    capacity  INT
);

CREATE TABLE allotments (
    allot_id   INT PRIMARY KEY AUTO_INCREMENT,
    room_id    INT,
    student    VARCHAR(100),
    allot_date DATE DEFAULT (CURRENT_DATE),
    vacated    BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);

INSERT INTO rooms (room_no, floor, capacity) VALUES
    ('101', 1, 3),
    ('102', 1, 2),
    ('201', 2, 4),
    ('204', 2, 2),
    ('301', 3, 3);
```

OUTPUT :-

```
===== Hostel Management =====
1. Add Room
2. Allot Room to Student
3. Vacate Student
4. View Room Status
5. Exit
Enter choice: 1
Enter Room No: 100
Enter Floor: 2
Enter Capacity: 4
Room added successfully!

===== Hostel Management =====
1. Add Room
2. Allot Room to Student
3. Vacate Student
4. View Room Status
5. Exit
Enter choice: 2
Enter Room No: 100
Enter Student Name: arju mallick
Room allotted successfully!
```

```
===== Hostel Management =====
1. Add Room
2. Allot Room to Student
3. Vacate Student
4. View Room Status
5. Exit
Enter choice: 4
Room 101 | Floor 1 | Capacity 3 | Occupied 0 | Available 3
Room 102 | Floor 1 | Capacity 2 | Occupied 0 | Available 2
Room 201 | Floor 2 | Capacity 4 | Occupied 0 | Available 4
Room 204 | Floor 2 | Capacity 2 | Occupied 0 | Available 2
Room 301 | Floor 3 | Capacity 3 | Occupied 0 | Available 3
Room 203 | Floor 3 | Capacity 12 | Occupied 0 | Available 12
Room 100 | Floor 2 | Capacity 4 | Occupied 1 | Available 3

===== Hostel Management =====
1. Add Room
2. Allot Room to Student
3. Vacate Student
4. View Room Status
5. Exit
Enter choice: 5
Thank you!
```

FOLDER STRUCTURE :